LangGraph Agent Graphs

Project Documentation

October 10, 2026

1 Overview

The three LangGraph applications in this project are built by `buildAdminAgentGraph()` in their respective files. All three compile the same `StateGraph(AdminAgentState)` topology. Only the tools bound to the LLM and the model differ:

File	LLM model	Tools
`DatabaseCreator.py`	`gpt-4.1-mini`	6
`FinancialGeneralInformationChatBot.py`	`gpt-4.1`	2
`OptimalBankDepositFinderAgent.py`	`gpt-5`	7

Shared structure

- **State**: `AdminAgentState` holds a single `messages` list, merged with `operator.add`.
- **Nodes**:
  - `AdminAgent` (`AdminAgentNode`): adds the system prompt once, then calls the tool-bound LLM.
  - `tool` (`ToolsNode`): runs every tool call in the last AI message and returns the `ToolMessage`s.
  - `ExitGuardrail` (`ExitGuardRailNode`): checks the final answer against the last human message, and replaces it with a refusal if it fails.
- **Conditional edges** (dashed):
  - From `__start__`, `inputGuardRailCondition` returns `continue` (input passes the guardrail) or `end`.
  - From `AdminAgent`, `where_to_go` returns `continue` (the AI message has `tool_calls`) or `end`.
- **Fixed edges** (solid): `tool` → `AdminAgent` and `ExitGuardrail` → `__end__`.

2 DatabaseCreator.py

Admin agent that fills the SQLite database from the RAG stores (model `gpt-4.1-mini`).

```
flowchart TD
    __start__([__start__])
    AdminAgent[AdminAgent]
    tool["tool<br/>UpdateSqlLiteTableStructure<br/>GetSqlLiteTablesStructure<br/>
    UpdateSqlLiteTableData<br/>GetSqlLiteTableData<br/>GetRequiredBankDepositDataFromTheRagTool
    <br/>GetRequiredMutualFundDataFromTheRagTool"]
    ExitGuardrail[ExitGuardrail]
    __end__([__end__])

    __start__ -. continue .-> AdminAgent
    __start__ -. end .-> __end__
    AdminAgent -. continue .-> tool
    AdminAgent -. end .-> ExitGuardrail
    tool --> AdminAgent
    ExitGuardrail --> __end__
```

Listing 1: Mermaid: DatabaseCreator.py

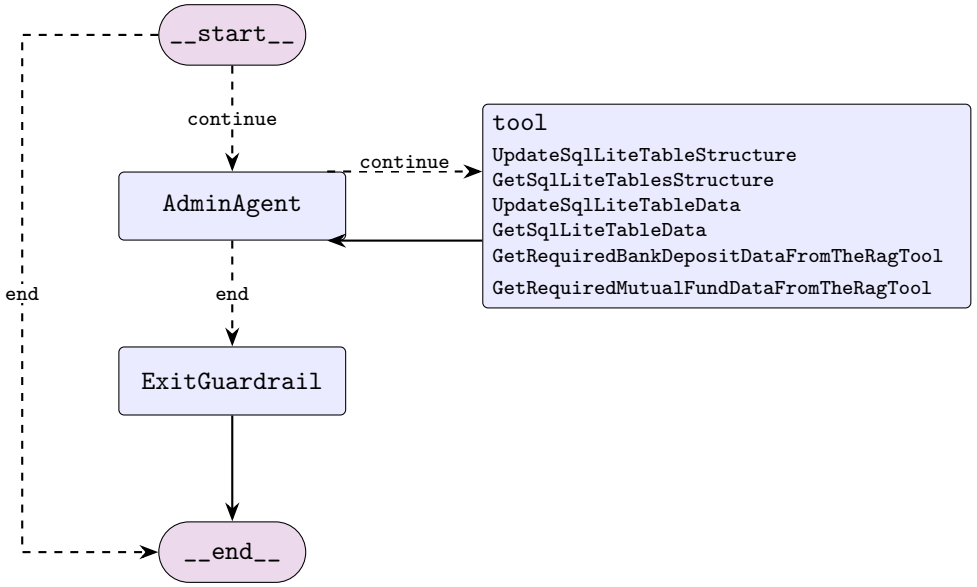


Figure 1: Graph of `DatabaseCreator.py`

3 FinancialGeneralInformationChatBot.py

Chatbot that answers general bank deposit and mutual fund questions from the RAG stores (model `gpt-4.1`).

```
flowchart TD
    __start__([__start__])
    AdminAgent[AdminAgent]
    tool["tool<br/>GetRequiredBankDepositDataFromTheRagTool<br/>
    GetRequiredMutualFundDataFromTheRagTool"]
    ExitGuardrail[ExitGuardrail]
    __end__([__end__])

    __start__ -. continue .-> AdminAgent
    __start__ -. end .-> __end__
    AdminAgent -. continue .-> tool
    AdminAgent -. end .-> ExitGuardrail
    tool --> AdminAgent
    ExitGuardrail --> __end__
```

Listing 2: Mermaid: FinancialGeneralInformationChatBot.py

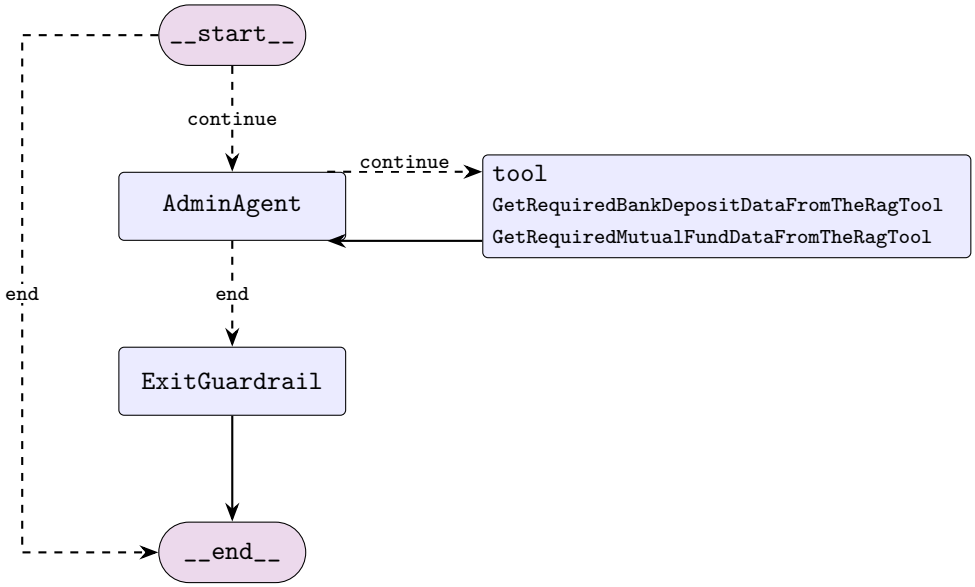


Figure 2: Graph of `FinancialGeneralInformationChatBot.py`

4 OptimalBankDepositFinderAgent.py

Agent that reads the deposit schemes from SQLite and finds the optimal fixed and recurring deposit sequence for a goal (model `gpt-5`). The tool `WriteMermaidDiagramPage` renders mermaid descriptions of the result as diagrams in an HTML page in the `output` folder.

```
flowchart TD
    __start__([__start__])
    AdminAgent[AdminAgent]
    tool["tool<br/>GetSqlLiteTablesStructure<br/>GetSqlLiteTableData<br/>
    OptimalActionFixedDepositTool<br/>OptimalActionRecurringDepositTool<br/>get_india_inflation
    <br/>GetFinalValueConsideringInflation<br/>WriteMermaidDiagramPage"]
    ExitGuardrail[ExitGuardrail]
    __end__([__end__])

    __start__ -. continue .-> AdminAgent
    __start__ -. end .-> __end__
    AdminAgent -. continue .-> tool
    AdminAgent -. end .-> ExitGuardrail
    tool --> AdminAgent
    ExitGuardrail --> __end__
```

Listing 3: Mermaid: OptimalBankDepositFinderAgent.py

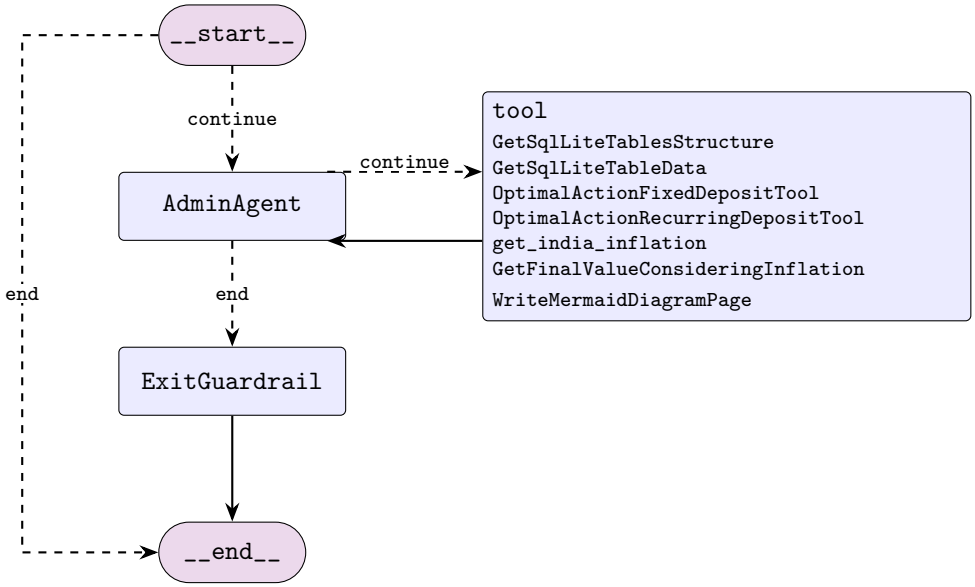


Figure 3: Graph of `OptimalBankDepositFinderAgent.py`